

รายงานสัปดาห์ที่ 5

โครงสร้างข้อมูล: Dictionaries & Sets (Data Structures: Dictionaries & Sets)

1. วัตถุประสงค์การเรียนรู้ (Learning Outcomes)

- จัดระเบียบข้อมูลด้วยคู่ key-value (dictionary) เพื่อการเรียกใช้ข้อมูลที่มีประสิทธิภาพ
- จัดการกลุ่มข้อมูลที่ไม่ซ้ำกันด้วย set และดำเนินการ set operations
- เลือกโครงสร้างข้อมูลที่เหมาะสม (list, tuple, dictionary หรือ set) ให้เหมาะสมกับปัญหาที่กำหนด

เกณฑ์การประเมิน (Trait Assessment): ประสิทธิภาพในการเรียกใช้ข้อมูล การจัดการข้อมูลที่ไม่ซ้ำกัน ความเข้าใจข้อดี-ข้อเสียของโครงสร้างข้อมูลแต่ละแบบ และการแก้ปัญหาอย่างเป็นเหตุเป็นผลโดยเลือกโครงสร้างข้อมูลที่เหมาะสม

2. สรุปเนื้อหาบทเรียน

2.1 Dictionary: โครงสร้างข้อมูลแบบคู่ Key-Value

dictionary คือกลุ่มข้อมูลแบบไม่เรียงลำดับ (unordered) และแก้ไขได้ (mutable) ที่เก็บข้อมูลเป็นคู่ key-value โดย key ต้องไม่ซ้ำกันและต้องเป็นชนิดข้อมูลที่ไม่เปลี่ยนแปลงไม่ได้ (immutable) เช่น string, number หรือ tuple ส่วน value เป็นชนิดข้อมูลใดก็ได้ สร้างด้วยเครื่องหมายปีกกา {} เช่น student = {"name": "Alice", "age": 20}

การเข้าถึงค่าในดิกชันนารีทำได้ผ่าน key โดยตรง (จะเกิด KeyError หากไม่พบ key) หรือใช้เมธอด get() ซึ่งจะคืนค่า None หรือค่า default ที่กำหนดไว้แทนที่จะเกิด error หากไม่พบ key

การเพิ่ม/แก้ไขทำได้ด้วยการกำหนดค่าให้ key โดยตรง ส่วนการลบใช้ del, pop(key) หรือ popitem() การวนลูปสามารถทำได้ทั้งวนผ่าน key (ค่าเริ่มต้น), วนผ่าน values() หรือวนผ่านคู่ key-value ด้วย items()

2.2 Set: โครงสร้างข้อมูลแบบไม่ซ้ำกันและไม่เรียงลำดับ

set คือกลุ่มข้อมูลแบบไม่เรียงลำดับที่แก้ไขได้ และสมาชิกทุกตัวต้องไม่ซ้ำกัน หากมีค่าซ้ำจะถูกลบออกโดยอัตโนมัติ ข้อควรระวังคือการสร้าง set ว่างต้องใช้ set() เนื่องจาก {} จะสร้างเป็น dictionary ว่างแทน

การเพิ่มสมาชิกใช้ add() ส่วนการลบใช้ remove() (จะเกิด KeyError หากไม่พบ), discard() (ไม่เกิด error หากไม่พบ) หรือ pop() ซึ่งจะลบสมาชิกแบบสุ่มเนื่องจาก set ไม่มีลำดับ

จุดเด่นของ set คือการดำเนินการเชิงเซตทางคณิตศาสตร์ ได้แก่ union() หรือ | (รวมสมาชิกทั้งหมด), intersection() หรือ & (สมาชิกร่วม), difference() หรือ - (สมาชิกที่มีเฉพาะเซตแรก) และ symmetric_difference() หรือ ^ (สมาชิกที่อยู่ในเซตใดเซตหนึ่งเท่านั้น)

รวมถึงการตรวจสอบ issubset(), issuperset() และ isdisjoint()

2.3 ตารางเปรียบเทียบโครงสร้างข้อมูลทั้งสี่แบบ

คุณสมบัติ	List / Tuple	Dictionary	Set
ลำดับ	เรียงลำดับ (ordered)	ไม่เรียงลำดับ	ไม่เรียงลำดับ
Mutability	list แก้ไขได้ / tuple แก้ไขไม่ได้	แก้ไขได้ (mutable)	แก้ไขได้ (mutable)
ค่าซ้ำ	อนุญาตให้ซ้ำได้	key ห้ามซ้ำ	สมาชิกห้ามซ้ำ
จุดเด่น	เก็บลำดับข้อมูล/พิกัดคงที่	ค้นหาข้อมูลด้วย key ได้เร็ว	ตรวจสอบสมาชิกและดำเนินการเชิงเซตได้เร็ว

3. Lab 5.1: Simple Contact Book Manager (Dictionary Focus)

3.1 วัตถุประสงค์ของแลป

พัฒนาโปรแกรมสมุดโทรศัพท์แบบ text-based โดยใช้ dictionary เพื่อฝึกปฏิบัติการเพิ่ม/แก้ไข ดู ลบ และแสดงรายชื่อผู้ติดต่อทั้งหมดผ่านระบบเมนูแบบวนซ้ำ (while loop)

3.2 สรุปขั้นตอนการทำงานของโปรแกรม

- สร้าง dictionary วางชื่อ contacts เพื่อเก็บข้อมูลผู้ติดต่อทั้งหมด
- Add/Update: รับชื่อ เบอร์โทร และอีเมลจากผู้ใช้ แปลงชื่อเป็นตัวพิมพ์เล็กด้วย .lower() เพื่อให้ค้นหาแบบไม่สนตัวพิมพ์เล็ก-ใหญ่ได้ (Challenge) แล้วเก็บเป็น contacts[name_key] = {"phone": ..., "email": ...} ซึ่งเป็นตัวอย่างของ nested dictionary
- View: ใช้ contacts.get(name_key) เพื่อดึงข้อมูลอย่างปลอดภัย หากไม่พบจะได้ None แทนที่จะเกิด KeyError
- List All: วนลูปด้วย contacts.items() เพื่อแสดงชื่อและรายละเอียดของผู้ติดต่อทุกคน
- Delete: ใช้ contacts.pop(name_key, None) เพื่อลบข้อมูลโดยไม่เกิด error หากไม่พบชื่อดังกล่าว
- เมนูทั้งหมดอยู่ใน while True: และจะออกจากโปรแกรมเมื่อผู้ใช้เลือกเมนู 5 ด้วยคำสั่ง break

3.3 โค้ดโปรแกรม (lab5_1.py)

```
# =====
# เจลลย: Lab 5.1 - Simple Contact Book Manager (Dictionary Focus) + Challenge
# =====

# --- ข้อที่ 1: สร้าง Dictionary วางสำหรับเก็บข้อมูลผู้ติดต่อ ---
contacts = {} #

while True:
    print("\n=== Contact Book Manager ===")
    print("1. Add/Update Contact")
    print("2. View Contact Details")
    print("3. List All Contacts")
    print("4. Delete Contact")
    print("5. Exit")

    choice = input("Enter choice (1-5): ")

    if choice == '1':
        name = input("Enter contact name: ")
        phone = input("Enter phone number: ")
        email = input("Enter email: ")

        # แปลงชื่อเป็นพิมพ์เล็กเพื่อให้เป็น case-insensitive (Challenge)
        name_key = name.lower()

        # --- ข้อที่ 2: เก็บข้อมูลลงใน contacts ---
        contacts[name_key] = {
            "phone": phone,
            "email": email
        } #
        print(f"Contact '{name}' saved successfully!")

    elif choice == '2':
        name = input("Enter contact name to view: ")
        name_key = name.lower() # แปลงชื่อเป็นพิมพ์เล็ก (Challenge)
```

```

# --- ข้อที่ 3: ใช้ .get() เพื่อดึงข้อมูล ---
info = contacts.get(name_key) #

if info:
    print(f"\nName: {name.title()}") # แสดงชื่อแบบตัวพิมพ์ใหญ่ตัวแรก
    print(f"Phone: {info['phone']}")
    print(f"Email: {info['email']}")
else:
    print("Contact not found.")

elif choice == '3':
    if not contacts:
        print("No contacts available.")
    else:
        print("\n--- All Contacts ---")
        # --- ข้อที่ 4: ใช้ .items() วนลูป ---
        for name_key, details in contacts.items(): #
            print(f"Name: {name_key.title()} | Phone: {details['phone']} | Email: {details['email']}")

elif choice == '4':
    name = input("Enter contact name to delete: ")
    name_key = name.lower() # แปลงชื่อเป็นพิมพ์เล็ก (Challenge)

    # --- ข้อที่ 5: ใช้ .pop() ลบข้อมูล ---
    removed_contact = contacts.pop(name_key, None) #[cite: 4]

    if removed_contact:
        print(f"Contact '{name}' deleted successfully!")
    else:
        print("Contact not found.")

elif choice == '5':
    print("Exiting Contact Book Manager. Goodbye!")
    # --- ข้อที่ 6: หยุดการทำงานลูป ---
    break #[cite: 4]

else:
    print("Invalid choice! Please enter a number from 1 to 5.")

```

3.4 ผลลัพธ์การรันโปรแกรม

ตัวอย่างการรันโดยเพิ่มผู้ติดต่อชื่อ Somsri พร้อมเบอร์โทรและอีเมล จากนั้นดูข้อมูล แสดงรายชื่อทั้งหมด ลบผู้ติดต่อ แล้วออกจากโปรแกรม:

```

=== Contact Book Manager ===
1. Add/Update Contact
2. View Contact Details
3. List All Contacts
4. Delete Contact
5. Exit
Enter choice (1-5): 1
Enter contact name: Somsri
Enter phone number: 081-1234567
Enter email: somsri@email.com
Contact 'Somsri' saved successfully!

=== Contact Book Manager ===
Enter choice (1-5): 2
Enter contact name to view: Somsri

```

```
Name: Somsri
Phone: 081-1234567
Email: somsri@email.com

=== Contact Book Manager ===
Enter choice (1-5): 3

--- All Contacts ---
Name: Somsri | Phone: 081-1234567 | Email: somsri@email.com

=== Contact Book Manager ===
Enter choice (1-5): 4
Enter contact name to delete: Somsri
Contact 'Somsri' deleted successfully!

=== Contact Book Manager ===
Enter choice (1-5): 5
Exiting Contact Book Manager. Goodbye!
```

3.5 คำถามท้ายแลปและคำตอบ

คำถาม: เหตุใดจึงใช้ `contacts.get(name_key)` แทนการเข้าถึงด้วย `contacts[name_key]` โดยตรงในเมนู View

คำตอบ: เพราะการเข้าถึงด้วย `contacts[name_key]` โดยตรงจะทำให้เกิด `KeyError` หากไม่พบชื่อดังกล่าวใน dictionary ซึ่งจะทำให้โปรแกรมหยุดทำงานกะทันหัน ในขณะที่ `get()` จะคืนค่า `None` แทน ทำให้สามารถตรวจสอบด้วย `if info:` แล้วแจ้งผู้ใช้งานว่า "Contact not found." ได้อย่างปลอดภัย

คำถาม: การแปลงชื่อผู้ติดต่อด้วย `.lower()` ก่อนเก็บและค้นหาช่วยแก้ปัญหาอะไร

คำตอบ: ช่วยให้การค้นหา แก่ไข และลบข้อมูลเป็นแบบ case-insensitive กล่าวคือผู้ใช้พิมพ์ชื่อด้วยตัวพิมพ์เล็กหรือใหญ่แบบใดก็ตามก็ยังค้นหาเจอผู้ติดต่อคนเดิม เพราะ key ที่เก็บจริงใน dictionary ถูกทำให้เป็นตัวพิมพ์เล็กทั้งหมดอย่างสม่ำเสมอ ตรงตามโจทย์ Challenge ของแลปนี้

คำถาม: เหตุใดค่า (value) ของแต่ละ key ใน `contacts` จึงถูกเก็บเป็น dictionary อีกชั้นหนึ่ง (nested dictionary)

แทนที่จะเป็นค่าธรรมดา

คำตอบ: เพราะผู้ติดต่อแต่ละคนมีข้อมูลหลายอย่างที่เกี่ยวข้งกัน (เบอร์โทรและอีเมล) การเก็บเป็น dictionary ย่อยภายใต้ key ของชื่อผู้ติดต่อ ทำให้จัดกลุ่มข้อมูลที่เกี่ยวข้องกันไว้ด้วยกันอย่างเป็นระเบียบ และเรียกใช้แต่ละฟิลด์ได้ง่ายด้วย `info['phone']` หรือ `info['email']`

4. Lab 5.2: Course Enrollment & Collaboration (Set Focus)

4.1 วัตถุประสงค์ของแลป

จำลองการลงทะเบียนเรียนของนักศึกษาในสองรายวิชาด้วย set แล้วใช้ set operations เพื่อวิเคราะห์ความสัมพันธ์ระหว่างรายชื่อนักศึกษาทั้งสองกลุ่ม

4.2 สรุปขั้นตอนการทำงานของโปรแกรม

- กำหนด set ของรหัสนักศึกษาสองชุด: python_students และ java_students
- Intersection: หานักศึกษาที่ลงทั้งสองวิชาด้วย python_students.intersection(java_students)
- Union: หารายชื่อนักศึกษาทั้งหมดแบบไม่ซ้ำกันด้วย python_students.union(java_students)
- Difference: หานักศึกษาที่ลงเฉพาะวิชา Python (ไม่ได้ลง Java) ด้วย python_students.difference(java_students)
- Symmetric Difference: หานักศึกษาที่ลงเพียงวิชาเดียว (ไม่ซ้ำกันทั้งสองวิชา) ด้วย python_students.symmetric_difference(java_students)
- สาธิตการลบข้อมูลซ้ำจาก list โดยแปลง list ที่มีรหัสซ้ำให้เป็น set แล้วแปลงกลับเป็น list อีกครั้ง

4.3 โค้ดโปรแกรม (lab5_2.py)

```
# =====
# เฉลย: Lab 5.2 - Course Enrollment & Collaboration (Set Focus)
# =====

python_students = {"S001", "S003", "S005", "S007", "S009"}
java_students = {"S002", "S003", "S006", "S007", "S010"}

print(f"Python Students: {python_students}")
print(f"Java Students: {java_students}\n")

# --- ข้อที่ 3: Intersection หานักศึกษาที่เรียนทั้ง 2 วิชา ---
both_courses = python_students.intersection(java_students) #[cite: 4]
print(f"Students in Both Courses: {both_courses}")

# --- ข้อที่ 4: Union หานักศึกษาทั้งหมดแบบไม่ซ้ำกัน ---
all_unique_students = python_students.union(java_students) #[cite: 4]
print(f"All Unique Students: {all_unique_students}")

# --- ข้อที่ 5: Difference หานักศึกษาที่เรียนเฉพาะ Python ---
only_python = python_students.difference(java_students) #[cite: 4]
print(f"Students Only in Python: {only_python}")

# --- ข้อที่ 6: Symmetric Difference หานักศึกษาที่เรียนแค่วิชาเดียว ---
exclusive_students = python_students.symmetric_difference(java_students) #[cite: 4]
print(f"Students in Only One Course: {exclusive_students}\n")

# --- ข้อที่ 7: การลบข้อมูลซ้ำออกจาก List ---
all_students_list = ["S001", "S003", "S005", "S001", "S002"]
print(f"Original List (with duplicates): {all_students_list}")

# แปลงเป็น set เพื่อลบตัวซ้ำ แล้วแปลงกลับเป็น list[cite: 4]
unique_students_list = list(set(all_students_list)) #[cite: 4]
print(f"Unique List (after removing duplicates): {unique_students_list}")
```

4.4 ผลลัพธ์การรันโปรแกรม

ผลลัพธ์จากการรันโปรแกรมด้วยชุดข้อมูล `python_students` และ `java_students` ตามที่โจทย์กำหนด (ลำดับสมาชิกใน `set` อาจแสดงไม่เรียงตามที่ป้อน เนื่องจาก `set` เป็นโครงสร้างข้อมูลแบบไม่เรียงลำดับ):

```
Python Students: {'S005', 'S009', 'S007', 'S003', 'S001'}
Java Students:   {'S002', 'S010', 'S007', 'S006', 'S003'}

Students in Both Courses: {'S007', 'S003'}
All Unique Students: {'S005', 'S009', 'S007', 'S001', 'S006', 'S002', 'S010', 'S003'}
Students Only in Python: {'S009', 'S005', 'S001'}
Students in Only One Course: {'S002', 'S010', 'S005', 'S009', 'S006', 'S001'}

Original List (with duplicates): ['S001', 'S003', 'S005', 'S001', 'S002']
Unique List (after removing duplicates): ['S001', 'S005', 'S003', 'S002']
```

4.5 คำถามท้ายแลปและคำตอบ

คำถาม: เหตุใดลำดับสมาชิกที่แสดงผลของ `set` จึงไม่ตรงกับลำดับที่กำหนดไว้ตอนสร้าง เช่น `python_students`

คำตอบ: เพราะ `set` เป็นโครงสร้างข้อมูลแบบไม่เรียงลำดับ (unordered) ภายในจะจัดเก็บสมาชิกตามค่า hash ของแต่ละสมาชิก ไม่ใช่ตามลำดับที่เขียนไว้ในโค้ด ดังนั้นเมื่อสั่ง `print` จึงอาจได้ลำดับที่แตกต่างจากที่พิมพ์ตอนสร้างเซต แต่สมาชิกและผลลัพธ์ของการดำเนินการเชิงเซตยังคงถูกต้องเสมอ

คำถาม: `difference()` ต่างจาก `symmetric_difference()` อย่างไร

คำตอบ: `python_students.difference(java_students)` จะคืนเฉพาะนักศึกษาที่อยู่ใน `python_students` แต่ไม่อยู่ใน `java_students` เพียงทิศทางเดียว ในขณะที่ `symmetric_difference()` จะคืนนักศึกษาที่อยู่ในเซตใดเซตหนึ่งเท่านั้นแต่ไม่อยู่ในทั้งสองเซต กล่าวคือรวมทั้งคนที่ลงทะเบียน Python และคนที่ลงทะเบียน Java เข้าด้วยกัน

คำถาม: การแปลง `list` ที่มีข้อมูลซ้ำเป็น `set` แล้วแปลงกลับเป็น `list` ช่วยแก้ปัญหาอะไร และมีข้อจำกัดอะไรบ้าง

คำตอบ: ช่วยลบข้อมูลที่ซ้ำกันออกจาก `list` ได้อย่างรวดเร็วโดยไม่ต้องเขียนลูปตรวจสอบเอง เนื่องจาก `set` จะเก็บเฉพาะสมาชิกที่ไม่ซ้ำกันโดยอัตโนมัติ ข้อจำกัดคือลำดับเดิมของ `list` อาจไม่ถูกรักษาไว้หลังแปลงกลับ เพราะ `set` ไม่มีการเรียงลำดับ จึงไม่เหมาะกับการกรณีที่ต้องการคงลำดับเดิมของข้อมูลไว้ด้วย

5. Activity 5.2: Word Frequency Counter (ส่วนเสริม)

นอกจาก Lab 5.1 และ 5.2 แล้ว ยังได้ทำ Activity 5.2 ซึ่งเป็นการประยุกต์ใช้ dictionary กับงานประมวลผลข้อความ โดยรับประโยคจากผู้ใช้งาน แปลงเป็นตัวพิมพ์เล็ก ตัดคำด้วย split() แล้วนับความถี่ของแต่ละคำด้วย dictionary พร้อมทั้งลบเครื่องหมายวรรคตอนออกก่อนนับ (Challenge) ด้วย word.strip('.,!?"')

```
# Activity 5.2 - Word Frequency Counter
# 1. รับข้อความประโยคจากผู้ใช้งาน
sentence = input("Enter a sentence: ") #[cite: 4]

# 2. แปลงข้อความให้เป็นตัวพิมพ์เล็กทั้งหมด
sentence_lower = sentence.lower() #[cite: 4]

# 3. แยกประโยคออกเป็นคำๆ
words = sentence_lower.split() #[cite: 4]

# 4. กำหนด Dictionary เปล่า
word_counts = {} #[cite: 4]

# 5. วนลูปอ่านคำแต่ละคำในลิสต์ words
for word in words: #[cite: 4]
    # ลบเครื่องหมายวรรคตอน (Challenge) [cite: 4]
    clean_word = word.strip('.,!?"') #[cite: 4]

    if clean_word == "": # ข้ามไปหากเป็นสตริงว่างหลังจากลบเครื่องหมาย
        continue

    # 6-8. ตรวจสอบว่าคำนั้นมีอยู่ใน Key แล้วหรือยัง
    if clean_word in word_counts: #[cite: 4]
        word_counts[clean_word] += 1 #[cite: 4]
    else:
        word_counts[clean_word] = 1 #[cite: 4]

# 9. แสดงผลลัพธ์
print("\n--- Word Frequencies ---")
for key, count in word_counts.items():
    print(f"'{key}': {count}")
```

ตัวอย่างผลลัพธ์จากประโยค "Python is fun, Python is powerful!":

```
Enter a sentence: Python is fun, Python is powerful!

--- Word Frequencies ---
'python': 2
'is': 2
'fun': 1
'powerful': 1
```

6. สรุปผลการทดลอง (Conclusion)

จากการทำแลปทั้งสองส่วนสรุปได้ว่า dictionary เหมาะสำหรับข้อมูลที่ต้องเรียกใช้ผ่านตัวระบุเฉพาะ (key) อย่างรวดเร็ว เช่น ข้อมูลผู้ติดต่อที่ต้องค้นหาด้วยชื่อ (Lab 5.1) ในขณะที่ set เหมาะสำหรับข้อมูลที่ต้องการความไม่ซ้ำกันและการเปรียบเทียบกลุ่มข้อมูล เช่น การหานักศึกษาที่ลงทะเบียนร่วมกันในหลายวิชา (Lab 5.2) การนำ dictionary มาใช้ในงานนับความถี่คำ (Activity 5.2)

ยังแสดงให้เห็นว่าโครงสร้างข้อมูลทั้งสองแบบสามารถผสมผสานกับ list และ tuple ที่เรียนมาก่อนหน้าได้

การเลือกโครงสร้างข้อมูลที่เหมาะสมกับลักษณะของปัญหานั้นตั้งแต่ต้น จึงเป็นปัจจัยสำคัญที่ทำให้โปรแกรมมีประสิทธิภาพและอ่านเข้าใจง่ายขึ้น